

Implement Client Script & UI Policy (Incident)

Comprehensive Project Documentation & Technical Implementation Guide

1. Project Team Information

This project documentation details the architecture, design, script development, and implementation guidelines for the **Implement Client Script & UI Policy (Incident)** project in ServiceNow. The project was collaboratively structured, developed, and tested by the following dedicated team members:

Role	Member Name	Responsibility Area
Team Leader	D Raniga	Project Oversight, Architecture Design, Code Review & Integration Testing
Team Member	K Seneha	Requirements Gathering, UI Policy Configuration & Field Visibility Rules
Team Member	R Sujitha	Client Script Development (onChange, onSubmit, onLoad) & Validation Logic

Role	Member Name	Responsibility Area
Team Member	K Swetha	Quality Assurance, Testing, Error Handling, and Comprehensive Documentation

Every configuration follows ServiceNow development best practices to ensure optimal client-side performance, data integrity, and enhanced user experience on the Incident form.

2. Executive Summary & Introduction

In enterprise service management platforms like ServiceNow, ensuring data accuracy and providing an intuitive user interface on core tables (such as the Incident table) is paramount. Service desk agents and end-users interact with incident forms daily, and enforcing business rules on the client side prevents invalid submissions before data reaches the database.

This project implements a combination of **Client Scripts** and **UI Policies** specifically tailored for the Incident table. While UI Policies provide a declarative, no-code mechanism to dynamically manage field visibility, mandatory status, and read-only states, Client Scripts handle complex asynchronous validations, dynamic dropdown manipulations, and form-level behaviors using JavaScript. Together, they create a responsive, error-free incident logging and management workflow.

3. Problem Statement & Project Objectives

3.1 Problem Statement

Uncontrolled incident forms often lead to common operational challenges:

- **Data Inconsistency:** Users submit tickets missing critical troubleshooting details or populate conflicting priority and impact values.

- **Cluttered User Interfaces:** Displaying irrelevant fields at all times overwhelms agents and increases data entry errors.
- **Delayed Validation:** Waiting for server-side validation during form submission degrades user experience and increases database load.

3.2 Project Objectives

1. **Dynamic Form Behavior:** Utilize UI Policies to dynamically show or hide fields based on category and subcategory selections.
2. **Real-time Field Validation:** Implement Client Scripts (onChange / onSubmit) to validate user input instantly (e.g., ensuring future dates are not selected or specific format constraints are met).
3. **Enforce Mandatory Requirements:** Dynamically make fields mandatory only when specific conditions are met (e.g., requiring closure notes when state changes to Closed).
4. **Optimize Performance:** Write lightweight, optimized client-side scripts adhering to ServiceNow coding standards (avoiding synchronous GlideRecord queries).

4. Technical Specifications & Design Architecture

The solution comprises two main configuration components implemented on the Incident [incident] table:

4.1 UI Policies Architecture

UI Policies provide declarative control over form fields without writing custom JavaScript code. The following key UI policies were implemented:

- **Policy 1: Make Close Notes Mandatory on Resolution/Closure**
 - *Condition:* State == Resolved OR State == Closed
 - *Action:* Make Close Code and Close Notes Mandatory and Visible.
- **Policy 2: Dynamic Hardware Details Visibility**
 - *Condition:* Category == Hardware
 - *Action:* Show Asset Tag and Model fields; otherwise hide them.

4.2 Client Scripts Architecture

Client Scripts execute JavaScript in the user's browser. The following scripts were engineered for this project:

Script Name	Type	Trigger Field	Purpose / Functionality
Validate Priority vs Impact	onChange	Impact / Urgency	Displays an info message or alert if an unusual impact/urgency combination is selected.
Auto-populate Caller Details	onChange	Caller	Fetches and displays caller location and department information upon caller selection.
Prevent Past Due Date Submission	onSubmit	Form Submission	Validates that any custom target resolution date is not set in the past.

5. Implementation & Code Snippets

Below are sample code implementations developed during the project lifecycle:

5.1 Sample onChange Client Script (Caller Info)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    // Call GlideAjax to fetch caller details asynchronously
    var ga = new GlideAjax('UserDetailAjax');
    ga.addParam('sysparm_name', 'getDepartment');
    ga.addParam('sysparm_user', newValue);
    ga.getXML(function(response) {
        var answer = response.responseXML.documentElement.getAttribute('answer');
        if (answer) {
            g_form.showFieldMsg('caller_id', 'Caller Department: ' + answer, 'info');
        }
    });
}
```

5.2 Sample onSubmit Client Script (Validation)

```
function onSubmit() {
    var state = g_form.getValue('state');
    var closeNotes = g_form.getValue('close_notes');

    // Check if incident is being resolved/closed without close notes
    if ((state == 6 || state == 7) && (closeNotes == '' || closeNotes == null)) {
        g_form.addErrorMessage('Please fill in Close Notes before resolving or closing the incident.');
```

```
        g_form.setMandatory('close_notes', true);
        return false; // Prevent form submission
    }
    return true;
}
```

6. Testing, Validation & Quality Assurance

Rigorous test scenarios were executed across developer instances to ensure seamless integration between UI Policies and Client Scripts without causing script conflicts or browser freezing.

6.1 Test Scenarios & Results

Test ID	Scenario Description	Expected Result	Actual Result	Status
TC01	Change Category to Hardware	Asset Tag and Model fields appear dynamically.	Fields appeared instantly.	Passed
TC02	Change Incident State to Resolved without Close Notes	Form submission is blocked with an error message.	Submission blocked successfully.	Passed
TC03	Select Caller on Incident Form	Caller department info message displays without page lag.	Ajax call resolved successfully.	Passed
TC04	Submit Form with valid data	Incident record is successfully	Record created without errors.	Passed

Test ID	Scenario Description	Expected Result	Actual Result	Status
		inserted into the database.		

7. Best Practices & Maintenance Guidelines

To ensure long-term stability and maintainability of the implemented Client Scripts and UI Policies, the team established the following guidelines:

- **Avoid Synchronous GlideRecord:** All database queries from client-side scripts were implemented using GlideAjax to prevent browser thread locking and UI freezing.
- **UI Policy Priority:** UI Policies were ordered correctly to prevent conflicting hide/show instructions on overlapping fields.
- **Isolate Logic:** Kept UI Policies separate for declarative behaviors and reserved Client Scripts strictly for conditional JavaScript logic and form validations.
- **Documentation & Naming Conventions:** Adopted standard naming prefixes (e.g., INC_) for all scripts and policies to simplify future administrative updates.

8. Conclusion

The **Implement Client Script & UI Policy (Incident)** project successfully demonstrates the power of combining ServiceNow's declarative UI Policies with robust client-side JavaScript. Led by **D Raniga** along with core contributions from team members **K Seneha**, **R Sujitha**, and **K Swetha**, the project streamlined the incident management interface, eliminated invalid form submissions, and elevated data quality across support workflows.

The comprehensive documentation and structured scripts provided in this report serve as a reliable deployment guide for future platform enhancements and administrative audits.